TIR Liability Coding — Plain-Language Overview

For the working session. No background in machine learning assumed.

What the program does

A coder reads a TIR description and decides which QPS codes it belongs under. This program reads the same description and suggests those codes, with a confidence attached to each.

It is built to **assist rather than replace**. Every suggestion carries a number saying how sure it is, and anything below a set bar is handed to a person rather than quietly guessed at. The bar is adjustable per field, so the team decides how much it wants to check.

It learned from **90,960 TIRs** — three years of the team's own decisions. Not every record carries every code, so each field learns from the ones that have it: all 90,958 for Metric Cat, 61,263 for Process Cat.

The four fields it codes

FIELD	HOW MANY CODES	WHAT IT IS
Metric Cat	7	The kind of quality issue
Process Cat	27	The discipline it belongs to
Process Sub	144	The sub-discipline
Process Level 3	543	The specific finding

The last two are chosen **inside** the one above them. If it decides a TIR is `HA Hanger`, it then picks only from the sub-codes that belong under hangers. A combination QPS would reject cannot be produced.

Those two counts are the codes it can actually learn. The full QPS lists are larger — 179 and 1,080 — but the rest have been used too rarely to learn from, which is improvement 3 below.

How it actually decides — the four methods

The program does not have one way of deciding. It has four, they compete, and the ones that do best are kept. Each works differently, which is the point: where they disagree is where the interesting cases are.

Support vector machine

Picture every description as a dot on a map, positioned by which words it contains. Descriptions using similar words sit near each other.

This method draws the dividing lines between categories, and puts each line as far as it can from the nearest examples on either side — so a borderline TIR still falls on the right side of it.

It handles having far more words than records, which is this problem exactly: 183,000 distinct words and word-fragments across 91,000 TIRs. **It is the method that usually wins here.**

Stochastic gradient

Draws the same kind of dividing line, but gets there differently. Instead of working out the best line in one go, it looks at a handful of TIRs at a time and nudges the line a little after each one, stopping when nudging stops helping.

Because it takes a different route it settles somewhere slightly different — which is the whole reason for keeping both. **On Process Cat it beat the support vector machine outright**, so both are now used together.

Logistic regression

Rather than drawing a line, it gives every word a score for and against every category. "Hanger" counts towards `HA Hanger`; "cable" counts against it. Add up the scores for the words in a description and you get how likely each category is.

Straightforward and quick, and it produces sensible probabilities. It has not yet beaten the other two on the full data.

Gradient boosting

Builds a long chain of small yes-or-no questions — *does the description mention a weld?* — each new question chosen to fix what the previous ones got wrong.

Very strong on many problems, and **the weakest of the four here.** Its way of working needs to pick individual words to ask about, and with 183,000 of them there are too many to choose from. It also needed so much memory that early runs were shut down by the machine; it now works from the 30,000 most useful words rather than all of them.

How the four are combined

1. All four learn the field.
2. Each is scored on TIRs it has never seen.
3. Anything that fails to beat the support vector machine is discarded.
4. If more than one survives, the program works out **how much to trust each**, again by testing on unseen TIRs.

That last step matters. Two methods surviving does not make them equally good. On Process Cat the second scored 2.4 points above the first, and until recently both were trusted equally — the program now works out the split and records both the old and new score so the difference is visible.

Measured on the last complete run:

FIELD	SUPPORT VECTOR	STOCHASTIC GRADIENT	LOGISTIC	BOOSTING	KEPT
Metric Cat	**73.0**	65.8	66.3	67.8	The first alone
Process Cat	71.1	**73.5**	70.6	65.4	The first two, blended

(The score counts every category equally, so a common one and a rare one weigh the same. It is deliberately harsher than "percent correct".)

What "confidence" means here

Every suggestion comes with a number between 0 and 100. It is a genuine probability: 80 means the program expects to be right about eight times in ten on TIRs it is that sure about.

That lets the team set a policy instead of a hope:

IF THE BAR IS SET AT	IT CODES AUTOMATICALLY	AND IS RIGHT	LEAVING FOR A CODER
Process Cat	**80%** of TIRs	**95%** of the time	20%
Metric Cat	**94%** of TIRs	**95%** of the time	6%

Process Sub and Process Level 3 **cannot reach 95%** at any setting. They top out near 92% and 94% even when only the most confident answers are kept. They can help a coder; they should not code unwatched.

What changed from the first prototype

	THE PROTOTYPE	NOW
Spreadsheets it reads	One layout only	Any of the three QPS produces
Fields coded	2	4
Do the codes fit together?	Not checked	Guaranteed — each level chosen inside the one above
Confidence	A number that could not be trusted	A real probability, set per field
Single TIRs	One at a time, nothing kept	Collected through a sitting, exported as one spreadsheet
Deciding which method to use	Three fixed, never compared	Four compete; only what earns its place is kept
Training data	One export	Both, with 12,321 duplicate records removed

Three things found in the data along the way

None of these were what anyone set out to look for.

The two spreadsheets overlap almost entirely. The smaller one is 99.4% contained in the larger. Because they name their columns differently, nothing noticed — the same TIRs would have been learned from twice and then used to mark the program's own homework.

A third of records had no Process Cat, and the prototype was learning "blank" as though it were a real category. It had become the single largest category in the training data.

Coders disagree with each other more than expected. Where the same description was coded more than once, the codes given differ:

FIELD	HOW OFTEN THE CODES CONFLICT
Metric Cat	32%
Process Cat	31%
Process Sub	43%
Process Level 3	52%

This is the most important finding in the project, and it is not a criticism of the coders — some of it is genuinely ambiguous TIRs. But **it sets a ceiling.** The program cannot be more consistent than the records it learned from, and it is already at that ceiling on every field. Getting better numbers now depends on agreeing the codes, not on better software.

Two questions worth answering up front

"Why does it only look at Description 1?"

For coding a TIR, it does not. The program reads **Description 1, Description 2 and the Doc Title** together. Adding the second and third was measured and kept: it improved the balanced score by 5.4 points, almost all of it on the rarer categories.

The confusion comes from a different number in this document — the one saying how often coders agree with each other. **That** measurement groups on Description 1 alone, and here is why.

To ask "was the same description coded the same way twice?" you first have to find descriptions that appear twice. Matching on all three fields makes near-identical TIRs look distinct, because Description 2 is blank on most records and the Doc Title varies between exports. The comparable set collapses:

FIELD	REPEATED DESCRIPTIONS FOUND USING DESCRIPTION 1	USING ALL THREE FIELDS
Metric Cat	1,974	179
Process Cat	932	83
Process Sub	897	80
Process Level 3	716	63

Eleven times fewer in every field — too few to say anything steady.

The trade-off is honest and stated: two TIRs sharing a Description 1 can legitimately be different events, so some of what is counted as disagreement is two coders correctly coding two different things. **That makes the agreement figures a floor, not a ceiling** — the real agreement is somewhat better than the numbers here. It is also why double-coding 200 TIRs deliberately would settle the question properly.

"Why build the code hierarchy from what coders actually did, rather than from the code numbering?"

The codes look like they nest. `HA Hanger` contains `HAPI Piping`, which contains `HAPIFK Fit/Alignment incorrect` — each code starts with its parent's code. It is tempting to use that rule to decide which sub-codes belong under which category.

We checked whether the records follow it:

LEVEL	CODES THAT NEST AS THE NUMBERING IMPLIES	RECORDS THAT DO NOT
Sub under Category	99.89%	66
Level 3 under Sub	95.51%	**2,555**

At the deepest level, **one record in twenty-two does not follow the rule.**

So the numbering describes what the taxonomy intended, and the records describe what was actually coded. Building from the numbering would do two things, both wrong:

- **Rule out real combinations.** Those 2,555 records were coded by people and presumably for a reason. A prefix rule would declare them impossible and the program could never suggest them.
- **Allow combinations nobody uses.** Any code pair whose numbering lines up would be permitted, including many that have never appeared in three years.

Building from observed pairs means the program can only suggest a combination that the team has actually used. It is worth knowing that the mismatch exists — **2,555 records disagreeing with the numbering is either a coding pattern worth understanding or a data-quality issue worth fixing**, and we cannot tell which from the data alone.

What would improve it

In order of what it is worth:

1. **Settle the conflicting codes.** 291 descriptions were given two different Process Cats. That list can be handed over as a spreadsheet — it is a day or two of work and it is the only thing that raises the ceiling.
2. **Write down the tie-breakers.** If experienced coders can name the five pairs of categories they always have to stop and think about, a single page of *"when it is both X and Y, code X"* helps the coders and the program equally.
3. **Retire the 537 Level 3 codes** used fewer than ten times in three years. They cannot be learned, and they are 537 options a coder has to scroll past.
4. **Confirm the Process Cat before the deeper codes are chosen.** Already in the app. Doing it lifts the sub-code from 81% to 90% — one click, worth more than every software change tried.

The honest summary

The program codes Metric Cat and Process Cat about as consistently as the team does, and can take roughly 80% of Process Cat off the queue at 95% accuracy.

The two deepest levels are genuinely hard — for people as much as for software — and should stay advisory.

Nothing further in the software will move these numbers much. What will is agreement about what the codes mean.